
UNIT IV- PL/SQL PROGRAMMIN

4.3

**Exception handling:
Predefined Exception,
User defined Exception.**

What is an Exception?

- An exception occurs when an **error** is discovered during the execution of a program that disrupts the normal operation of the program.
- There are many possible causes of exceptions:
 - a user makes a spelling mistake while typing;
 - a program does not work correctly;
 - an advertised web page does not exist; and so on.
- **An error condition during a program execution is called an exception in PL/SQL.**
- PL/SQL supports programmers to catch such conditions using EXCEPTION block in the program and an appropriate action is taken against the error condition.

Exceptions in PL/SQL

- If No data is found for our query, we will get such message.

`ORA-01403: no data found`

- This type of error in PL/SQL is called an exception.
 - When an exception occurs, we say that the exception has been “raised”.
 - When an exception is raised, the rest of the execution section of the PL/SQL block is not executed.
- There are two types of exceptions:
 - System-defined (Predefined) Exceptions
 - User-defined Exceptions

What is an Exception Handler?

- An exception handler is code that defines the recovery actions to be performed when an exception is raised (i.e. when an error occurs).
- When writing code,
 - programmers need to anticipate the types of errors that can occur during execution of that code.
 - In a sense, exception handlers allow programmers to "bulletproof" their code.
- What types of errors may programmers want to account for by using an exception handler?
 - System errors (e.g., a hard disk is full)
 - Data errors (e.g., trying to duplicate a primary key value)
 - User action errors (e.g., data entry error)
 - Many other possibilities !

Why is Exception Handling Important?

- Some reasons include:
 - Protects the user from errors (frequent errors may frustrate the user and/or cause the user to quit the application)
 - Protects the database from errors (data can be lost or overwritten)
 - Major errors take a lot of system resources (if a mistake is made, correcting the mistake may be costly; users may frequently call the help desk for assistance with errors).
 - Code is more readable because error-handling routines can be written in the same block in which the error occurred.

Syntax for Exception Handling

- The General Syntax for exception handling is as follows.
- Here we can list down as many as exceptions we want to handle.
- The **default exception** will be handled using ***WHEN others THEN***

DECLARE

<declarations section>

BEGIN

<executable command(s)>

EXCEPTION

<exception handling goes here >

WHEN exception1 THEN

exception1-handling-statements

WHEN exception2 THEN

exception2-handling-statements

.....

WHEN others THEN

exception3-handling-statements

END;

Example –EXCEPTION HANDLING

SET VERIFY OFF

DECLARE

 e_id emp.empno%type := &EMPNO;

 e_name emp.ename%type;

 e_sal emp.sal%type;

BEGIN

 SELECT ename, sal INTO e_name, e_sal FROM emp WHERE empno = e_id;

 DBMS_OUTPUT.PUT_LINE ('Name: ' || e_name);

 DBMS_OUTPUT.PUT_LINE ('Salary: ' || e_sal);

EXCEPTION

 WHEN no_data_found THEN

 dbms_output.put_line('No such Employee!');

 WHEN others THEN

 dbms_output.put_line('Error!');

END;

/

Example - explained

When the above code is executed at SQL prompt, it produces the following result:

Enter value for empno: 7777

No such Employee!

PL/SQL procedure successfully completed.

- The above program displays the name and salary of an employee whose ID is given.
- Since there is no customer with ID value 7777 in our database, the program raises the run-time exception `NO_DATA_FOUND`, which is captured in `EXCEPTION` block.

Raising Exceptions

- Exceptions are raised by the database server automatically whenever there is any internal database error, but exceptions can be raised explicitly by the programmer by using the command **RAISE**.
- Following is the simple syntax of raising an exception:

DECLARE

exception_name EXCEPTION;

BEGIN

IF condition THEN

RAISE exception_name;

END IF;

EXCEPTION

WHEN exception_name THEN

statement;

END;

We can use this syntax in raising Oracle standard exception or any user-defined exception.

User-defined Exceptions

- PL/SQL allows us to define our own exceptions according to the need of the program.
- A user-defined exception must be declared and then raised explicitly, using either a **RAISE** statement or the procedure **DBMS_STANDARD.RAISE_APPLICATION_ERROR**.
- The syntax for declaring an exception is:

DECLARE

<exception_name> EXCEPTION;

Example

```
DECLARE
    e_id emp.empno%type := &e_id;
    e_name emp.ename%type;
    e_sal emp.sal%type;
    -- user defined exception
    ex_invalid_id EXCEPTION;

BEGIN
    IF e_id <= 0 THEN
        RAISE ex_invalid_id;
    ELSE
        SELECT ename, sal INTO e_name, e_sal FROM emp WHERE empno = e_id;
        DBMS_OUTPUT.PUT_LINE ('Name: ' || e_name);
        DBMS_OUTPUT.PUT_LINE ('Salary: ' || e_sal);
    END IF;
EXCEPTION
    WHEN ex_invalid_id THEN
        DBMS_OUTPUT.PUT_LINE ('ID must be greater than zero');
    WHEN no_data_found THEN
        dbms_output.put_line('No such Employee!');
    WHEN others THEN
        dbms_output.put_line('Error!');

END;
/
```

Output:

Enter value for e_id: -10

old 2: e_id emp.empno%type := &e_id;

new 2: e_id emp.empno%type := -10;

ID must be greater than zero

PL/SQL procedure successfully completed.

Pre-defined Exceptions

- PL/SQL provides many pre-defined exceptions, which are executed when any database rule is violated by a program.
- For example, the predefined exception `NO_DATA_FOUND` is raised when a `SELECT INTO` statement returns no rows.
- The following table lists few of the important pre-defined exceptions:

Exception	Oracle Error	SQLCODE	Description
<code>ACCESS_INTO_NULL</code>	06530	-6530	It is raised when a null object is automatically assigned a value.
<code>DUP_VAL_ON_INDEX</code>	00001	-1	It is raised when duplicate values are attempted to be stored in a column with unique index.
<code>INVALID_CURSOR</code>	01001	-1001	It is raised when attempts are made to make a cursor operation that is not allowed, such as closing an unopened cursor.
<code>INVALID_NUMBER</code>	01722	-1722	It is raised when the conversion of a character string into a number fails because the string does not represent a valid number.

Pre-defined Exceptions

Exception	Oracle Error	SQLCODE	Description
LOGIN_DENIED	01017	-1017	It is raised when a program attempts to log on to the database with an invalid username or password.
NO_DATA_FOUND	01403	+100	It is raised when a SELECT INTO statement returns no rows.
TOO_MANY_ROWS	01422	-1422	It is raised when a SELECT INTO statement returns more than one row.
VALUE_ERROR	06502	-6502	It is raised when an arithmetic, conversion, truncation, or size-constraint error occurs.
ZERO_DIVIDE	01476	1476	It is raised when an attempt is made to divide a number by zero.

Handling Exceptions with PL/SQL

- Below is another example. The select statement in the block is retrieving the **name** of **SCOTT**. For this, we have used pattern matching operator:

```
DECLARE
    name VARCHAR2(15);
BEGIN
    SELECT ename INTO name FROM emp WHERE ename like 'S%';
    DBMS_OUTPUT.PUT_LINE('Employee name is : ' || name);
END;
```

/

- However, an exception is raised because more than one name starting with 'S' exists in the data.

DECLARE

*

ERROR at line 1:

ORA-01422: exact fetch returns more than requested number of rows

ORA-06512: at line 4

- Here, we have used pattern matching operator with full matching name:

```
DECLARE
    name VARCHAR2(15);
BEGIN
    SELECT ename INTO name FROM emp WHERE ename like 'SCOTT';
    DBMS_OUTPUT.PUT_LINE('Employee name is : ' || name);
END;
```

/

- Provides the output as follows.
Employee name is : SCOTT
PL/SQL procedure successfully completed.

Handling Exceptions with PL/SQL

- We can handle such exceptions in our PL/SQL block so that our program completes successfully. For example:
- The following code includes a handler for a predefined Oracle server error called **TOO_MANY_ROWS**.

```
DECLARE
    name VARCHAR2(15);
BEGIN
    SELECT ename INTO name FROM emp WHERE ename like 'S%';
    DBMS_OUTPUT.PUT_LINE('Employee name is : ' || name);
    EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Your select statement retrieved
            multiple rows. Consider using a cursor.');
```

END;

/

Output:

Your select statement retrieved multiple rows. Consider using a cursor.

PL/SQL procedure successfully completed.

Handling Exceptions – ZERO_DIVIDE

- In this example, a PL/SQL program attempts to divide by 0.
- The **ZERO_DIVIDE** predefined exception is used to trap the error in an exception-handling routine.

```
DECLARE
    v_num integer;
BEGIN
    v_num := 1000/0;
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Your are attempting to divide by
            zero. ');
END;
/
```

Output:

Your are attempting to divide by zero.

PL/SQL procedure successfully completed.

Using RAISE statement to trap user-defined exception

- In this example, the deptno 100 does not exist, so no rows are updated in the **dept** table.
- The RAISE statement is used to explicitly raise an exception and display an error message, returned by the SQLERRM built-in function, and an error code, returned by the SQLCODE built-in function.

```
DECLARE
    v_deptno NUMBER := 500;
    v_name VARCHAR2 (20) := 'Testing';
    e_invalid_dept EXCEPTION;
BEGIN
    UPDATE dept
    SET dname = v_name
    WHERE deptno = v_deptno;
    IF SQL%NOTFOUND THEN
        RAISE e_invalid_dept;
    END IF;
    ROLLBACK;
    EXCEPTION
    WHEN e_invalid_dept THEN
        DBMS_OUTPUT.PUT_LINE ('No such department');
        DBMS_OUTPUT.PUT_LINE (SQLERRM);
        DBMS_OUTPUT.PUT_LINE (SQLCODE);
END;
/
```

Output:

```
No such department
User-Defined Exception
1
PL/SQL procedure successfully completed.
```